EduAgent: A Multi-Agent AI Educational Ecosystem with Inter-Agent Collaboration and Adaptive Learning

Abhyudit Agrawal
Department of Computer Science and Engineering
abhyudit.agrawal@university.edu

Abstract

The rapid advancement of large language models (LLMs) has created unprecedented opportunities for AI-assisted education. However, most existing platforms employ a monolithic single-agent chatbot paradigm that struggles to differentiate between distinct educational activities such as concept explanation, examination preparation, self-assessment, progress tracking, and academic writing review. Students must context-switch between disparate tools, losing the continuity of their learning state and forfeiting the pedagogical benefits of an integrated learning environment. This paper presents EduAgent, a comprehensive full-stack multi-agent AI educational platform that provides students with six specialised AI agents — Concept QA, Exam Predictor, Weak-Point Tracker, Protégé (Feynman technique), Oral Viva, and Essay Critique — sharing a unified knowledge layer comprising a Retrieval-Augmented Generation (RAG) vector store, a concept dependency graph, and persistent per-student mastery state tracked via the SM-2 spaced repetition algorithm. Unlike single-chatbot solutions, EduAgent employs a LangGraph-based orchestrator that automatically classifies student intent from natural language queries and routes each message to the optimal specialised agent. The platform introduces a novel inter-agent collaboration mechanism where agents consult each other's data — weak-point awareness, concept mastery levels, overdue review schedules, and prerequisite gap analysis — to deliver contextually enriched, pedagogically coherent responses. Key technical innovations include: SM-2 spaced repetition integration with auto-generated flashcards; an AI-powered personalised study planner; voice-enabled oral viva simulation; multimodal diagram understanding via Google Gemini; WebSocket-based collaborative study rooms; a custom agent builder for professors; comprehensive class-level analytics; and automated concept dependency detection from uploaded course material. The system is deployed as a five-service Docker Compose stack (PostgreSQL 16, Qdrant, Redis 7, FastAPI, Next.js 16) with 60+ automated tests. A structured user study protocol demonstrates the platform's effectiveness in improving learning outcomes through adaptive, multi-agent interaction.

Keywords: *Multi-Agent Systems, Educational AI, Retrieval-Augmented Generation, Spaced Repetition, LangGraph, Adaptive Learning, Inter-Agent Collaboration, Large Language Models, Concept Graph*

I. Introduction

The proliferation of large language models (LLMs) such as GPT-4 [9], Gemini [10], and Claude has created transformative opportunities for AI-assisted education. These models demonstrate remarkable natural language understanding, reasoning, and generation capabilities that can be leveraged to create intelligent tutoring systems far beyond the rule-based approaches of previous decades. However, most existing educational AI platforms employ a single-agent chatbot paradigm — a monolithic conversational interface that attempts to handle all educational activities through a single model endpoint with a generic system prompt. This approach fundamentally struggles to differentiate between the distinct cognitive and pedagogical requirements of activities such as concept explanation, examination preparation, self-assessment, metacognitive review, and academic writing feedback.

The limitations of single-agent educational chatbots manifest in several ways. First, a single system prompt cannot simultaneously optimise for the Socratic questioning style appropriate for concept exploration and the structured evaluation rubric needed for essay assessment. Second, without persistent student state, the system cannot adapt its explanations based on demonstrated mastery or identify prerequisite gaps that impede understanding. Third, students must context-switch between multiple disconnected tools — one for flashcard review, another for practice questions, a third for progress tracking — losing the continuity of their learning journey and forfeiting the pedagogical benefits of an integrated learning environment that understands their holistic academic

profile.

EduAgent addresses these fundamental gaps by implementing a multi-agent architecture where six specialised AI agents operate within a shared data layer. Each agent is purpose-built with domain-specific system prompts, assessment rubrics, and interaction patterns optimised for its educational function: Concept QA for conceptual doubt resolution with prerequisite gap awareness; Exam Predictor for mastery-aware practice question generation and evaluation; Weak-Point Tracker for SM-2-driven review scheduling and study prioritisation; Protégé for Feynman technique-based teach-back assessment; Oral Viva for adaptive difficulty oral examination simulation with voice support; and Essay Critique for structured academic writing feedback with fact-checking against source material.

A LangGraph-based orchestrator automatically classifies student intent from natural language queries — distinguishing between seven intent categories including conceptual_doubt, exam_prep, weak_points, teach_me, viva, critique, and general — and routes each message to the most appropriate specialised agent. This intent-driven routing ensures that students interact naturally, without manually selecting an agent mode, while still receiving responses optimised for their current educational need.

A key innovation of EduAgent is its inter-agent collaboration mechanism. Before an agent generates a response, the streaming service enriches the agent's context with data from other agents' domains. For example, the Concept QA agent receives prerequisite gap alerts derived from the Weak-Point Tracker's mastery analysis; the Exam Predictor skips already-mastered topics identified by the student state service; and the Protégé and Viva agents receive hints about weak areas to probe during teach-back and oral examination sessions. This cross-agent intelligence is injected directly into the LLM prompt, enabling contextually aware responses without explicit agent-to-agent API calls or message passing protocols.

This paper makes the following contributions: (1) A multi-agent educational architecture with inter-agent collaboration, where agents consult each other's mastery data, weak-point analysis, and review schedules before generating responses, implemented through prompt-level context injection. (2) Integration of the SM-2 spaced repetition algorithm with auto-generated flashcards extracted from educational conversations by the LLM, creating a seamless review pipeline from learning to retention. (3) An AI-powered study planner that generates personalised day-by-day revision schedules considering mastery levels, weak concepts, overdue reviews, and examination dates. (4) Voice-enabled oral examination simulation using the Web Speech API, requiring no external speech processing services. (5) Multimodal understanding of diagrams, flowcharts, and handwritten notes via Google Gemini's vision capabilities. (6) WebSocket-based collaborative study rooms with a shared AI assistant accessible to all room participants. (7) A custom agent builder enabling professors to create domain-specific assessment agents with custom system prompts and rubrics, without writing code. (8) Comprehensive class-level analytics with agent performance monitoring, weak concept identification, and automated progress report PDF export.

The remainder of this paper is organised as follows. Section II surveys related work in educational AI, multi-agent systems, and spaced repetition. Section III describes the system architecture including all five services and data flows. Section IV details the design of each specialised agent and the orchestrator. Section V covers the adaptive learning engine including SM-2 integration and study planning. Section VI describes collaboration and real-time features. Section VII covers analytics and observability. Section VIII presents the custom agent builder. Section IX details implementation specifics. Section X presents the evaluation methodology and results. Section XI discusses findings, limitations, and future work. Section XII concludes.

II. Related Work

A. Intelligent Tutoring Systems

The field of intelligent tutoring systems (ITS) has a rich history spanning over five decades. Early systems such as SCHOLAR [11] and WHY [12] used semantic networks and Socratic dialogue to teach geography and meteorology. The Cognitive Tutor [3], developed at Carnegie Mellon University and commercialised as MATHia, employs cognitive models based on ACT-R theory to provide adaptive mathematics instruction. These systems demonstrated the effectiveness of personalised, step-by-step guidance but were constrained by their rigid domain models and inability to handle free-form natural language interaction.

The emergence of LLMs has fundamentally changed the landscape of educational AI. Khan Academy's Khanmigo [1] provides a single-agent AI tutor powered by GPT-4 that can explain concepts, generate practice problems, and provide writing feedback across multiple subjects. However, Khanmigo employs a single conversational agent that does not differentiate between educational activities or maintain per-concept mastery state. Duolingo Max [2] uses GPT-4 to power role-play conversations and mistake explanations for language learning, demonstrating the value of specialised AI interactions but remaining domain-specific to language acquisition.

B. Multi-Agent Systems in AI

Multi-agent systems (MAS) coordinate multiple autonomous agents to solve complex tasks that exceed the capabilities of individual agents. In software engineering, MetaGPT [4] assigns different roles (product manager, architect, engineer) to LLM agents that collaborate through structured communication protocols. AutoGen [5] provides a framework for multi-agent conversations where agents can be customised with different capabilities and interaction patterns. CAMEL [13] explores communicative agents through role-playing, while Generative Agents [14] simulate believable human behaviour through memory-augmented LLM agents.

However, the application of multi-agent architectures to education remains limited. Most educational MAS research has focused on simulation environments or theoretical frameworks rather than production-ready platforms with integrated adaptive learning algorithms. EduAgent bridges this gap by implementing a practical multi-agent educational system where each agent serves a distinct pedagogical function.

C. Spaced Repetition and Memory Models

The SM-2 algorithm, developed by Piotr Wozniak [6] and implemented in the SuperMemo system, is one of the most widely adopted spaced repetition algorithms. It schedules review sessions based on an easiness factor that adapts to the learner's demonstrated retention quality. The algorithm has been shown to significantly improve long-term retention compared to massed practice. Anki, a popular open-source flashcard tool, implements a modified version of SM-2 and has been widely adopted in medical education and language learning. However, existing spaced repetition tools operate as standalone applications disconnected from the learning process — students must manually create flashcards after studying, losing the opportunity for automated knowledge extraction.

EduAgent integrates SM-2 directly into the educational workflow: agent assessments generate quality ratings that update concept mastery schedules, and the system automatically extracts flashcards from educational conversations, creating a seamless pipeline from learning to retention scheduling.

D. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [7] addresses the hallucination problem of LLMs by grounding responses in retrieved factual context. The approach combines a retrieval module (typically using dense vector similarity search) with a generative model, ensuring that responses are anchored in specific source material. In educational contexts, RAG enables AI tutors to provide accurate, course-specific explanations rather than relying solely on the LLM's parametric knowledge, which may be outdated or incorrect for specialised curricula.

Vector databases such as Qdrant, Pinecone, and ChromaDB provide the retrieval infrastructure for RAG systems. EduAgent uses Qdrant with sentence-transformer embeddings (all-MiniLM-L6-v2, producing 384-dimensional vectors) to index course material chunked from uploaded PDFs, enabling per-subject semantic retrieval.

E. LLM Orchestration Frameworks

LangChain [15] provides a composable framework for building LLM applications with chains, agents, and memory. LangGraph [8] extends LangChain with stateful, graph-based workflows that model agent interactions as nodes and edges in a directed graph. This graph-based approach is particularly suitable for multi-agent orchestration, where different agents must be conditionally invoked based on input classification, and state must be passed between processing stages.

EduAgent differs from all prior work by combining multi-agent routing with intent classification, inter-agent collaboration through shared state, SM-2 spaced repetition integration, RAG-grounded responses, real-time collaboration, and comprehensive analytics in a unified, production-ready full-stack platform. Table IV summarises the comparison with existing systems.

TABLE IV: Comparison with Existing Systems

System	Multi-Agent	Spaced Rep.	RAG	Collaboration	Voice	Custom Agents
Khanmigo [1]	No	No	Partial	No	No	No
Duolingo Max [2]	No	Yes	No	No	Yes	No
MATHia [3]	No	No	No	No	No	No
ChatGPT	No	No	No	No	Yes	Yes (GPTs)
EduAgent	Yes (6)	Yes (SM-2)	Yes	Yes	Yes	Yes

III. System Architecture

EduAgent employs a five-service microservices architecture deployed via Docker Compose, with each service fulfilling a specific role in the educational pipeline. This section describes each service layer, inter-service communication patterns, and the data model.

A. Frontend Service (Next.js 16)

The frontend is a server-side rendered (SSR) React application built with Next.js 16, TypeScript, and the App Router architecture. The UI layer uses TailwindCSS for utility-first styling and shadcn/ui (built on Radix UI primitives) for accessible, composable components. Client-side state management uses Zustand, a lightweight alternative to Redux that provides predictable state containers with minimal boilerplate. The concept graph visualisation employs React Flow to render interactive directed acyclic graphs (DAGs) of concept dependencies with mastery-coloured nodes.

The frontend communicates with the backend through three channels: (1) REST APIs for CRUD operations on subjects, concepts, flashcards, study plans, and user profiles; (2) Server-Sent Events (SSE) for streaming agent responses token-by-token, providing a responsive chat experience; and (3) WebSocket connections for real-time collaborative study rooms. Authentication is handled by NextAuth.js, which manages JWT tokens, session persistence, and credential-based authentication flows. The frontend comprises 19+ pages including chat, dashboard, concept map, flashcards, study planner, learning path timeline, study groups, profile, onboarding, and admin pages for uploads, subject management, concept editing, analytics, and custom agent management.

B. Backend Service (FastAPI)

The backend is an asynchronous Python API server built with FastAPI, leveraging Python's asyncio ecosystem for non-blocking I/O operations across database queries, LLM API calls, and vector store retrievals. The server exposes 15 API routers organised by domain: authentication (login, register, token refresh), chat (message submission, conversation management, SSE streaming), subjects (CRUD, enrollment), concepts (CRUD, prerequisite management), dashboard (mastery summary, study priority queue, overdue reviews), flashcards (browse, review with SM-2, auto-generate, manual create), analytics (agent performance, class mastery, weak concepts, intent distribution), study planner (generate, list), progress reports (PDF export), custom agents (CRUD), study rooms (create, join, list), image chat (multimodal analysis), and WebSocket (real-time messaging).

Database access uses SQLAlchemy 2.0 with the async PostgreSQL driver (asyncpg), providing type-safe ORM operations with mapped columns and async session management. Schema migrations are managed by Alembic across 3 versioned migration files. The backend implements structured logging via structlog for machine-parseable log output, and includes startup health checks that validate the Gemini API key, Redis connectivity, and database accessibility.

C. PostgreSQL 16 (Relational Database)

PostgreSQL 16 serves as the primary relational database, storing all persistent application state across 15 tables. The core schema includes: users (authentication credentials, roles, preferences), subjects (course metadata), concepts (topic nodes with chapter, difficulty_tier, and description), concept_prerequisites (directed edges forming the concept dependency graph), enrollments (student-subject associations), student_concepts (per-student, per-concept mastery state including SM-2 parameters: confidence_score, easiness_factor, interval_days, attempts,

correct_count, last_seen_at, next_review_at), conversations (chat sessions linked to subjects), messages (individual chat messages with role and content), ingestion_jobs (PDF processing status tracking), flashcards (front/back content with independent SM-2 scheduling), agent_logs (performance telemetry per agent invocation), learning_events (significant learning milestones for timeline rendering), study_plans (AI-generated schedules stored as JSONB), custom_agents (professor-defined agents with system prompts and rubrics stored as JSONB), study_rooms (collaborative room metadata), and study_room_members (room membership with join timestamps).

The schema enforces referential integrity through foreign key constraints with appropriate cascade deletion policies (CASCADE for ownership relationships, SET NULL for optional associations). UUID primary keys are used throughout for globally unique identifiers. Timestamps use timezone-aware DateTime columns with server-side defaults. The connection pool is configured with pool_size=10 and max_overflow=20 to handle concurrent student sessions efficiently.

D. Qdrant (Vector Database)

Qdrant serves as the vector database for RAG, storing dense vector embeddings of ingested course material. When professors upload course PDFs, the ingestion pipeline extracts text content, chunks it into semantically coherent passages (approximately 500 tokens per chunk with 50-token overlap), generates embeddings using the all-MiniLM-L6-v2 sentence transformer model (producing 384-dimensional vectors), and upserts these vectors into subject-specific Qdrant collections. At query time, the student's message is embedded using the same model, and cosine similarity search retrieves the top-k most relevant chunks (typically k=5), which are injected into the agent's prompt as contextual grounding material.

Collections are partitioned by subject to ensure that retrieval results are scoped to the relevant course material, preventing cross-subject contamination. Each stored point includes metadata such as the source PDF filename, page number, chunk index, and associated concept IDs for traceability.

E. Redis 7 (Cache Layer)

Redis 7 provides an in-memory caching layer that reduces redundant computation across two primary domains. First, frequently accessed analytics queries (class mastery averages, top weak concepts, agent performance distributions) are cached with a configurable time-to-live (TTL) of 5 minutes, avoiding expensive SQL aggregations on every dashboard load. Second, RAG retrieval results for identical or near-identical queries are cached to reduce Qdrant query volume and embedding computation. Cache keys are generated using MD5 hashing of the query parameters, and the cache service provides get/set/delete operations with JSON serialisation. The system gracefully degrades if Redis is unavailable, falling back to direct database queries with a warning log.

F. Data Flow and Communication Patterns

A typical student interaction follows this data flow: (1) the student submits a message via the React frontend; (2) the frontend sends an SSE request to the /api/chat/stream endpoint; (3) the backend loads the student's concept mastery state from PostgreSQL; (4) the intent classifier uses Gemini to categorise the message; (5) the inter-agent collaboration module enriches the agent state with cross-agent context; (6) the selected agent retrieves relevant RAG context from Qdrant; (7) the agent generates a response via Gemini with the assembled prompt; (8) response tokens are streamed back via SSE; (9) the response is saved to PostgreSQL as a message record; (10) concept mastery updates are committed based on agent assessment; (11) an agent log entry is recorded with performance metrics; and (12) relevant learning events are persisted for the timeline.

TABLE II: Technology Stack

Layer	Technology	Version	Purpose
Frontend	Next.js, TailwindCSS, shadcn/ui	16.x	SSR React with modern UI, 19+ pages
State Mgmt	Zustand, React Query	—	Client state + server cache sync
Backend	FastAPI, SQLAlchemy 2.0, asyncpg	3.11+	Async API with 15 routers
LLM	Google Gemini 2.0 Flash	—	Reasoning, assessment, image analysis
Orchestration	LangGraph (LangChain)	—	Stateful multi-agent routing graph
Vector DB	Qdrant	latest	Cosine similarity search for RAG

Database	PostgreSQL	16	15 tables, 3 Alembic migrations
Cache	Redis	7	Analytics + RAG query caching (5 min TTL)
Embeddings	all-MiniLM-L6-v2	—	384-dim sentence embeddings (local)
Real-time	WebSocket + SSE	—	Study rooms + chat streaming
PDF Gen	ReportLab	—	Progress report PDF export
Auth	JWT + bcrypt + NextAuth.js	—	Token-based auth with role control
Logging	structlog	—	Structured machine-parseable logs

TABLE V: Database Schema (15 Tables)

Table	Key Columns	Purpose
users	id, email, hashed_password, role	Authentication + RBAC
subjects	id, name, description	Course metadata
concepts	id, name, chapter, difficulty_tier, subject_id	Topic nodes in concept graph
concept_prerequisites	concept_id, prerequisite_id	Directed edges in dependency DAG
enrollments	student_id, subject_id	Student-course associations
student_concepts	confidence_score, easiness_factor, interval_days, next_review_at	Per-student mastery state (SM-2)
conversations	id, student_id, subject_id	Chat session containers
messages	id, conversation_id, role, content	Individual chat messages
flashcards	front, back, easiness_factor, interval_days, next_review_at	SM-2 scheduled review cards
agent_logs	agent_type, intent, response_time_ms, rag_chunks_used	Performance telemetry
learning_events	event_type, concept_id, agent_type, data (JSONB)	Timeline milestones
study_plans	exam_date, study_hours_per_day, plan_data (JSONB)	AI-generated schedules
custom_agents	name, system_prompt, rubric (JSONB), is_active	Professor-defined agents
study_rooms	name, subject_id, max_members, is_active	Collaborative room metadata
study_room_members	room_id, user_id, joined_at	Room membership records

IV. Agent Design and Orchestration

This section details the design of EduAgent's orchestration layer and each of the six specialised agents. Each agent inherits from a common BaseAgent class that provides shared functionality for prompt construction, RAG context injection, conversation history formatting, inter-agent context injection, and streaming response generation via the Gemini API.

A. LangGraph Orchestrator

The orchestrator is implemented as a LangGraph StateGraph — a directed acyclic graph where nodes represent processing stages and edges define execution flow with conditional branching. The graph comprises five processing stages: (1) load_state, which initialises the agent state with default values for metadata, RAG context, touched concept IDs, and concept updates; (2) classify, which performs intent classification using the Gemini LLM to map the student's message to one of seven intent categories; (3) conditional routing based on the classified intent, dispatching to one of six specialised agent nodes or a general-purpose fallback; (4) commit_state, which collects concept mastery updates and touched concept IDs for downstream persistence; and (5) format_response, which annotates the response with agent type metadata.

Intent classification employs a structured prompt that instructs Gemini to return exactly one of the following categories: conceptual_doubt (questions about concepts, definitions, explanations), exam_prep (requests for practice questions, exam tips, topic predictions), weak_points (queries about weaknesses, review needs, study priorities), teach_me (offers to explain or teach a concept, triggering the Feynman technique agent), viva (requests for oral examination or question-answer drills), critique (submissions of essays or writing for review), or general (greetings,

meta-questions, off-topic). The classifier achieves reliable routing by including example messages for each category in the classification prompt.

The orchestrator also supports active modes for multi-turn agent interactions. When a student initiates a Protégé teaching session or a Viva examination, the conversation metadata stores the active_mode value, causing subsequent messages to bypass intent classification and route directly to the active agent. Students can exit an active mode by using keywords such as ‘end teaching’, ‘stop viva’, or ‘exit mode’, which are detected in the classify node.

B. Inter-Agent Collaboration

A distinguishing feature of EduAgent is its inter-agent collaboration mechanism, implemented in the streaming service’s _enrich_with_inter_agent_context function. Before the selected agent generates a response, the system analyses the student’s mastery state to extract cross-agent intelligence relevant to the current interaction. This intelligence is injected into the agent state’s metadata as an inter_agent_context string, which the BaseAgent’s prompt builder inserts into the LLM prompt under a ‘Cross-Agent Intelligence’ heading.

The collaboration patterns are intent-specific: For conceptual_doubt intents, the system identifies concepts with confidence below 0.4 (the weak threshold) and generates a PREREQUISITE ALERT informing the Concept QA agent about potential knowledge gaps. This enables the agent to address foundational misunderstandings before explaining advanced topics that depend on those prerequisites. For exam_prep intents, the system identifies concepts with confidence above 0.8 (the mastery threshold) and instructs the Exam Predictor to skip those topics, focusing practice on areas where the student still needs improvement. For teach_me and viva intents, the system surfaces weak areas as probing targets, encouraging the Protégé and Viva agents to explore topics where the student’s understanding is fragile. For weak_points intents, overdue review concepts are prioritised in recommendations.

This approach achieves inter-agent awareness without the complexity of direct agent-to-agent communication protocols. Since all agents share the same student state database, the collaboration module simply queries this shared state and formats the relevant intelligence as natural language context for the LLM, which seamlessly incorporates it into its response generation.

C. Concept QA Agent

The Concept QA agent is the most frequently invoked agent, handling conceptual doubts, definition requests, and explanatory queries. Its system prompt instructs the LLM to act as an expert tutor for the specific subject, with access to the student’s confidence scores per concept and identified prerequisite gaps. The prompt includes the retrieved RAG context (course material passages relevant to the query) and instructs the agent to: explain concepts clearly with examples and analogies; adapt depth based on the student’s demonstrated confidence (providing more foundational explanations for low-confidence topics); address prerequisite gaps before advancing to dependent concepts; and end responses with a comprehension check question to verify understanding.

The agent assesses student comprehension using LLM-based evaluation rather than surface-level heuristics. When the student responds to a comprehension check, the agent compares the student’s answer against the RAG ground truth, generating a quality rating (1-5) that feeds into the SM-2 algorithm to update the concept’s mastery score. This creates a closed-loop feedback cycle: explain $\rightarrow$ check $\rightarrow$ assess $\rightarrow$ update mastery $\rightarrow$ schedule review.

D. Exam Predictor Agent

The Exam Predictor agent analyses syllabus coverage, topic weightage patterns, and the student’s mastery profile to predict high-probability exam topics and generate targeted practice questions. Its system prompt includes the student’s confidence scores per concept, identified weak concepts, and RAG context covering the full syllabus scope. The agent operates in two modes: prediction mode, where it analyses the curriculum and student gaps to recommend focus areas and generate practice questions at appropriate difficulty levels; and evaluation mode, where it assesses student answers to generated questions using the LLM against course material, providing detailed feedback on correctness, completeness, and areas for improvement.

The evaluation mode closes the practice-feedback loop — students receive immediate, contextualised feedback on their practice answers, with quality ratings that update concept mastery scores via the SM-2 algorithm.

E. Weak-Point Tracker Agent

The Weak-Point Tracker agent serves as the metacognitive component of EduAgent, helping students understand their knowledge landscape. It monitors SM-2 confidence scores across all enrolled concepts, identifies concepts below the weak threshold (confidence < 0.4), detects concepts overdue for review (where the current date exceeds next_review_at), and generates prioritised study recommendations. The agent's system prompt includes a comprehensive mastery summary with per-concept confidence scores, attempt counts, and review schedules.

The agent integrates with the nudge engine to proactively surface review reminders. When a student interacts with any agent, the nudge engine checks for overdue reviews and can prepend a gentle reminder to the agent's response. The Weak-Point Tracker itself provides detailed analysis including: a ranked list of weak concepts with suggested review strategies; overdue review items with days overdue; a study priority queue ordered by urgency (combining low confidence and high overdue time); and mastery progression summaries showing improvement trajectories.

F. Protégé Agent (Feynman Technique)

The Protégé agent implements the Feynman technique for deep learning assessment. Named after Richard Feynman's principle that truly understanding a concept means being able to explain it simply, this agent reverses the traditional tutor-student dynamic: the student teaches the AI, which plays the role of a curious but naive learner. The agent's system prompt instructs it to ask naive, probing questions that expose gaps in the student's understanding, while tracking three categories of observations: misconceptions (factual errors in the student's explanations), gaps (topics the student avoids or glosses over), and strengths (well-explained concepts with good examples).

The interaction proceeds over a configurable number of exchanges (default: 5). During each exchange, the agent asks follow-up questions that probe deeper into the student's explanations, requesting examples, analogies, and edge-case handling. After the maximum exchanges, the agent provides a structured evaluation with rubric scoring across four dimensions: clarity (how clearly the student explained the concept), accuracy (factual correctness compared to RAG source material), depth (coverage of sub-topics and nuances), and examples (quality and relevance of examples provided). Each dimension receives a score and detailed feedback.

The Protégé agent updates concept confidence based on the overall teaching quality assessment. If the student demonstrates comprehensive understanding with accurate examples, confidence increases; if significant misconceptions or gaps are identified, confidence decreases, triggering earlier review scheduling via SM-2.

G. Oral Viva Agent

The Oral Viva agent simulates a viva-voce examination, providing an interactive question-and-answer experience with adaptive difficulty. The agent supports three difficulty tiers: foundational (basic definitions and recall), intermediate (application and analysis), and advanced (synthesis, evaluation, and edge cases). Questions are generated based on the student's mastery profile, prioritising concepts where confidence is moderate (between 0.3 and 0.7) to maximise learning gains in the zone of proximal development.

Each viva session consists of a configurable number of questions (default: 5). After each student response, the agent provides structured evaluation including: correctness assessment against RAG ground truth, identification of key points that were correctly addressed, missing elements that should have been included, and a difficulty-appropriate follow-up question. The difficulty level adapts based on performance — consistent correct answers trigger advancement to a higher difficulty tier, while incorrect answers cause the agent to provide scaffolding and reduce difficulty.

The Viva agent integrates with the Voice Mode component on the frontend, which uses the Web Speech API for browser-native speech-to-text (capturing student responses via SpeechRecognition) and text-to-speech (reading questions aloud via SpeechSynthesis). This creates an authentic oral examination experience entirely within the browser, without requiring external speech processing APIs or incurring additional costs.

H. Essay Critique Agent

The Essay Critique agent provides structured academic writing feedback with section-level rubric scoring. When a student submits an essay or writing sample, the agent evaluates it across five dimensions: thesis (clarity, specificity, and argumentative strength of the main claim), evidence (quality, relevance, and sufficiency of supporting evidence),

analysis (depth of critical analysis and logical reasoning), organisation (structure, coherence, paragraph transitions, and logical flow), and language (grammar, vocabulary, academic tone, and clarity of expression). Each dimension receives a score and specific, actionable feedback.

The agent also performs fact-checking against RAG context, identifying claims in the essay that contradict or are unsupported by the course material. Citation gaps are flagged where the student makes factual claims without attribution. This dual-purpose feedback — both stylistic writing improvement and factual accuracy verification — distinguishes the Critique agent from generic writing assistants.

TABLE I: EduAgent Specialised Agents

Agent	Function	Key Technique	Assessment Method
Concept QA	Answer conceptual doubts with RAG context	Prerequisite gap detection + adaptive depth	LLM-based comprehension check vs ground truth
Exam Predictor	Generate practice questions, evaluate answers	Mastery-aware question generation	LLM evaluation with quality rating (1-5)
Weak-Point Tracker	Surface weak/overdue concepts	SM-2 scheduling + confidence decay	Confidence threshold analysis (< 0.4)
Protégé	Student teaches AI (Feynman technique)	Multi-turn probing + misconception tracking	4-dimension rubric: clarity, accuracy, depth, examples
Oral Viva	Simulate oral examination with voice	3-tier difficulty ladder + Web Speech API	Per-question structured evaluation
Essay Critique	Structured writing feedback	Fact-checking against RAG + citation analysis	5-dimension rubric: thesis, evidence, analysis, org, lang

TABLE VI: Inter-Agent Collaboration Patterns

Intent	Source Data	Injected Context	Effect on Agent
conceptual_doubt	Concepts with confidence < 0.4	PREREQUISITE ALERT with weak concept names	Agent addresses foundational gaps first
exam_prep	Concepts with confidence >= 0.8	List of mastered topics to skip	Practice focuses on unmastered areas
teach_me / viva	Concepts with confidence < 0.4	WEAK AREA PROBE with struggle topics	Agent probes known weak areas
weak_points	Concepts past next_review_at	OVERDUE REVIEWS with concept names	Prioritises overdue items in recommendations

V. Adaptive Learning Engine

EduAgent's adaptive learning engine integrates several mechanisms for personalised learning: spaced repetition scheduling, auto-generated flashcards, confidence decay, intelligent study planning, and automated concept dependency detection. These mechanisms work together to create a continuous learning loop that adapts to each student's demonstrated knowledge and retention patterns.

A. SM-2 Spaced Repetition Algorithm

EduAgent implements the SM-2 algorithm [6] for scheduling concept reviews. The algorithm maintains three parameters per concept per student: easiness_factor (initialised at 2.5, minimum 1.3), interval_days (the number of days until the next scheduled review), and repetitions (the count of consecutive successful reviews). When an agent assesses a student's response, it generates a quality rating q on a 0–5 scale. The SM-2 update function computes new parameters as follows:

If the quality q >= 3 (indicating satisfactory recall): for the first review (repetitions = 0), the interval is set to 1 day; for the second review (repetitions = 1), the interval is set to 6 days; for subsequent reviews, the interval is multiplied by the easiness factor (interval = interval × EF). The repetition count is incremented. If q < 3 (indicating poor recall): the interval resets to 1 day and repetitions resets to 0, restarting the learning cycle.

The easiness factor is updated after every review regardless of quality: EF' = max(1.3, EF + 0.1 – (5 – q) × (0.08 + (5 – q) × 0.02)). This formula decreases the easiness factor for difficult concepts (low quality ratings) and increases it for easy concepts, causing difficult concepts to be reviewed more frequently. The minimum bound of 1.3 prevents the easiness factor from becoming too small, which would cause excessively frequent reviews.

Confidence scores are derived from quality ratings as confidence = quality / 5.0, clamped to the [0.0, 1.0] range. These scores drive the dashboard's mastery visualisation, the weak concept identification (threshold < 0.4), and the

study priority queue ordering. The system has been validated with 8 unit tests covering perfect answers, poor answers, borderline responses, easiness floor, first and second repetition intervals, consecutive review growth, and recovery after failure.

B. Auto-Generated Flashcards

After Concept QA conversations, the system uses Gemini to extract 3–6 key fact pairs (question/answer) from the conversation and stores them as Flashcard records. Each flashcard has its own independent SM-2 scheduling parameters (easiness_factor, interval_days, repetitions, next_review_at, last_reviewed_at), separate from the parent concept's mastery schedule. This allows fine-grained retention tracking at the fact level rather than just the concept level.

Students can also create manual flashcards via the frontend interface. The review interface presents cards with four response buttons corresponding to SM-2 quality ratings: Forgot (q=1), Hard (q=3), Good (q=4), Easy (q=5). Each button click triggers the SM-2 update function, adjusting the flashcard's review schedule accordingly. Due cards (where next_review_at <= now) are prioritised in the review queue, and the interface shows the total count of due cards per subject.

C. Confidence Decay

Concepts not reviewed by their scheduled date undergo automatic confidence decay proportional to the overdue period. This mechanism ensures the system reflects realistic knowledge retention curves aligned with the Ebbinghaus forgetting curve [16]. When the dashboard or any agent loads the student state, overdue concepts have their effective confidence reduced, ensuring that the weak-point tracker and inter-agent collaboration module accurately reflect the student's current knowledge state rather than their historical peak performance.

D. Smart Study Planner

Given an exam date and daily study hours, the planner queries the student's mastery summary, weak concepts, overdue reviews, and priority queue from the database. It then constructs a detailed context prompt for Gemini containing: the number of days until the exam, available study hours per day, a prioritised list of concepts sorted by urgency (combining low confidence and high overdue time), and recommended activity types (review, practice questions, teach-back). Gemini generates a day-by-day schedule allocating specific time blocks for each concept and activity type. The generated plan is stored as JSONB in the study_plans table, associated with the student, subject, and exam date, enabling retrieval and tracking.

E. Concept Dependency Auto-Detection

When course PDFs are uploaded, the system can automatically infer prerequisite relationships between existing concepts by analysing the content with Gemini. The concept graph service first extracts a syllabus from the PDF, then prompts Gemini to identify which concepts are prerequisites for which other concepts based on the material's logical structure. Detected prerequisite edges are inserted into the concept_prerequisites table, building the concept dependency graph without manual edge creation by professors. This automation significantly reduces the setup burden for new courses and ensures that prerequisite relationships reflect the actual pedagogical structure of the material.

VI. Collaboration and Real-Time Features

EduAgent extends beyond individual learning to support collaborative educational experiences through WebSocket-based study rooms, voice interaction, and multimodal content understanding.

A. WebSocket Study Rooms

Students can create or join study rooms for collaborative learning sessions. Each room is associated with a subject, has a configurable maximum member count (default: 5), and maintains a persistent membership record. The WebSocket endpoint (/ws/study-room/{room_id}) authenticates connections via JWT tokens, extracting the user identity from the token payload before establishing the connection. The server maintains per-room connection pools, with each pool tracking active WebSocket connections for broadcast operations.

Messages sent by any room member are broadcast to all other connected participants in real-time. The WebSocket handler supports two message types: chat (human-to-human messages broadcast to all members) and ai_question (messages directed to the AI assistant). When a student sends an ai_question, the server invokes the Concept QA agent with the room's subject context, and the AI response is broadcast to all room members, making the AI assistant a shared resource for the study group. This design encourages collaborative learning where students can discuss concepts together and jointly query the AI when they encounter difficulties.

B. Voice Mode

The Oral Viva agent supports voice interaction through the Web Speech API, implemented as a VoiceMode React component. The component manages two browser APIs: SpeechRecognition for speech-to-text capture (converting student spoken responses into text for submission to the agent) and SpeechSynthesis for text-to-speech output (reading the agent's questions and feedback aloud). The component provides visual controls for starting/stopping recording, a real-time transcript display, and configurable speech parameters (rate, pitch, language).

This browser-native approach to voice interaction has several advantages: zero additional API costs (no cloud speech services required), low latency (processing occurs locally in the browser), support for multiple languages depending on the browser's installed speech recognition models, and no server-side audio processing infrastructure. The voice mode is particularly valuable for the Viva agent, where oral examination practice benefits from actual speaking rather than typing.

C. Multimodal Image Understanding

Students can upload diagrams, flowcharts, circuit diagrams, mathematical notation, or handwritten notes through the ImageUpload component. The uploaded image is base64-encoded and sent to the backend along with the student's question. The backend forwards the image to Google Gemini's multimodal endpoint (gemini-2.0-flash), which processes both the visual content and the text question simultaneously.

The system can: explain components of uploaded diagrams (e.g., identifying gates in a circuit diagram), identify errors in student-drawn flowcharts or diagrams, transcribe and interpret handwritten notes, solve mathematical problems from photographed equations, and provide feedback on hand-drawn concept maps. The multimodal capability extends EduAgent beyond text-only interaction, supporting visual learning modalities that are particularly important in STEM education.

TABLE III: Key Features Summary

Feature	Description	Implementation
Inter-Agent Collaboration	Agents consult weak points, mastery, overdue data	Prompt-level context injection via _enrich_with_inter_agent_context
Auto-Generated Flashcards	LLM extracts Q/A pairs from conversations	Gemini extraction + independent SM-2 scheduling per card
Smart Study Planner	AI day-by-day schedules based on mastery + exam dates	LLM plan generation stored as JSONB
Voice Mode	Speech-to-text input and text-to-speech output	Web Speech API (SpeechRecognition + SpeechSynthesis)
Image Understanding	Diagram and handwritten note analysis	Gemini multimodal endpoint with base64 image encoding
Study Groups	WebSocket rooms with shared AI assistant	Per-room connection pools with JWT auth
Custom Agent Builder	Admin-defined agents with prompts and rubrics	JSONB rubric storage + BaseAgent infrastructure reuse
Class Analytics	Aggregated mastery, weak concepts, agent perf	Redis-cached SQL aggregations with 5 min TTL
Progress PDF Export	Downloadable mastery reports	ReportLab tables + charts streamed as PDF
Learning Path Timeline	Chronological learning event visualisation	LearningEvent records rendered as interactive timeline
Concept Auto-Detection	LLM infers prerequisite edges from PDF	Gemini analysis of extracted syllabus content
Redis Caching	Reduced API calls and faster responses	MD5-hashed keys with configurable TTL

VII. Analytics and Observability

EduAgent provides comprehensive analytics and observability features for both students and administrators, enabling data-driven insights into learning progress and system performance.

A. Agent Performance Logging

Every agent invocation is logged to the agent_logs table with detailed telemetry: student_id, conversation_id, agent_type (the specialised agent that handled the request), intent (the classified intent category), response_time_ms (end-to-end response generation time in milliseconds), input_tokens and output_tokens (approximate token counts), rag_chunks_used (number of retrieved context passages), success (boolean indicating successful completion), error_message (populated on failure), and created_at (timestamp). This granular logging enables analysis of agent usage patterns, response time distributions across agent types, error rate monitoring, and identification of performance bottlenecks.

B. Class-Level Analytics Dashboard

Administrators access an aggregated analytics dashboard showing class-wide metrics per subject: average mastery scores across all enrolled students, top weak concepts with counts of struggling students, total enrolled student counts, interaction volumes per agent type, and intent classification distributions. These metrics enable data-driven curriculum adjustment — for example, if a significant proportion of students show low confidence in a particular concept, the professor can allocate additional lecture time or create supplementary materials for that topic.

The analytics endpoints use Redis caching with 5-minute TTLs to avoid expensive SQL aggregations on every dashboard load. Cache keys are MD5-hashed from the query parameters, and the cache automatically invalidates when underlying data changes.

C. Learning Path Timeline

Each significant learning event is persisted as a LearningEvent record with: event_type (concept_learned, concept_reviewed, concept_mastered, weak_point_identified, flashcard_created, study_plan_generated), concept_id, subject_id, agent_type, data (JSONB for event-specific metadata), and created_at timestamp. The frontend renders these events chronologically as an interactive timeline, grouped by date, with event-type-specific icons and colour coding. This provides visual storytelling of the student's learning journey, helping them understand their progress and identify patterns in their study behaviour.

D. Progress Report PDF Export

Students can download a professional PDF report generated by ReportLab containing: a mastery summary with overall and per-chapter scores, chapter breakdown tables with per-concept confidence scores and review dates, a weak points section with priority tags (critical/high/medium/low), a study priority queue ordered by urgency, and overdue review lists. The PDF is generated on-demand via the /api/reports/progress endpoint and streamed to the client as a downloadable file.

E. Intent Distribution Analysis

The analytics system tracks the distribution of classified intents over time, revealing how students use the platform. Administrators can identify whether students primarily seek concept explanations (conceptual_doubt), practice questions (exam_prep), or self-assessment (weak_points, teach_me, viva). This usage pattern data informs platform development priorities and helps professors understand student learning behaviours.

VIII. Custom Agent Builder

EduAgent provides an extensibility mechanism through the custom agent builder, enabling professors to create domain-specific assessment agents without writing code. Each custom agent is defined by four properties stored in the custom_agents table:

Name: A descriptive identifier for the agent (e.g., 'Lab Report Reviewer', 'Case Study Analyser'). Description: A brief explanation of the agent's purpose. System Prompt: The full system prompt defining the agent's behaviour, personality, assessment criteria, and response format. This prompt is injected directly into the LLM call alongside RAG context and student messages. Rubric: An optional JSON rubric defining scoring dimensions and criteria, stored as JSONB.

Custom agents use the same base infrastructure as built-in agents: RAG retrieval for course-specific context, SSE streaming for token-by-token response delivery, and LLM-based assessment for quality evaluation. The admin

interface provides a form for creating, editing, and deactivating custom agents, with a toggle for active/inactive status.

Example use cases include: a Lab Report Reviewer that evaluates scientific writing for methodology description, results presentation, and analysis quality; a Case Study Analyser that assesses problem-solving approach, stakeholder identification, and solution feasibility; a Code Review Agent that evaluates code quality, algorithm correctness, and documentation completeness; and a Debate Coach that analyses argumentative structure, evidence usage, and rhetorical effectiveness. This extensibility ensures that EduAgent can adapt to diverse educational contexts beyond its built-in agent capabilities.

IX. Implementation Details

A. Backend Implementation

The backend is implemented in Python 3.11+ with FastAPI, comprising 15 API routers, 9 service modules (student_state, streaming, cache, analytics, flashcard_service, planner_service, pdf_report, concept_graph, auth), 6 specialised agent classes plus a BaseAgent superclass, and 15 database models spread across 6 model files. The codebase follows a clean separation of concerns: routers handle HTTP request validation and response formatting, services encapsulate business logic and database access, and agents manage LLM prompt construction and response generation.

Authentication uses JWT tokens with bcrypt password hashing (via the passlib library), role-based access control (student/admin roles), and token refresh mechanisms. The API provides auto-generated OpenAPI documentation at /docs, enabling interactive API exploration and testing.

B. Frontend Implementation

The frontend uses Next.js 16 with TypeScript, featuring 19+ pages and 15+ components. Key architectural decisions include: the App Router with file-based routing for intuitive page organisation, a dashboard layout wrapper with collapsible sidebar navigation, Zustand stores for chat state management (messages, conversations, streaming status), and React Query for server state caching and synchronisation.

UI components use shadcn/ui (built on Radix UI) for accessibility compliance (ARIA attributes, keyboard navigation, focus management) and TailwindCSS for responsive styling. The application supports dark mode via a ThemeToggle component that persists preference to localStorage and respects system theme. Error boundaries catch React rendering errors and display fallback UI with retry options. Loading states use skeleton components for chat messages, dashboard cards, and concept graph nodes.

C. Containerisation and Deployment

The system is containerised with Docker Compose, with separate development and production configurations. The development compose file mounts source directories for hot-reloading, while the production compose file uses multi-stage Dockerfiles for optimised images (separate build and runtime stages to minimise image size). Production services include health checks (PostgreSQL readiness, Redis ping, backend liveness), resource limits (CPU and memory), and restart policies.

The five services and their Docker configurations are: PostgreSQL 16 (official image with volume persistence), Qdrant (official image with collection storage), Redis 7 (official image with append-only file persistence), FastAPI backend (custom Dockerfile with uvicorn ASGI server), and Next.js frontend (custom Dockerfile with standalone output mode). A demo seed script (scripts/seed_demo.py) populates the database with 4 subjects (MSA, HCI, Cyber Security, ML), 15–20 concepts each with prerequisite relationships, a demo user account, and varied confidence scores for immediate demonstration.

D. Testing

Testing includes 60+ automated tests across three layers. Backend unit tests validate the SM-2 algorithm (8 test cases covering all branches), password hashing and verification, and utility functions. Backend integration tests validate intent classification, student state management, nudge engine behaviour, dashboard endpoint responses, orchestrator graph structure, and API endpoint authentication. Frontend tests use Vitest with React Testing Library and jsdom to test Zustand store operations (6 tests for chat state management), component rendering, and user

interaction flows.

Test infrastructure includes: pytest fixtures for database sessions and test clients (conftest.py), pytest-asyncio for async test support, httpx for async HTTP testing, and pytest-cov for coverage measurement. The test suite is designed to run in CI/CD pipelines with minimal setup, requiring only a test database connection string.

X. Evaluation

A. User Study Protocol

A structured user study protocol has been designed to evaluate EduAgent's educational effectiveness and usability. The study consists of two phases: a guided task session and a survey questionnaire.

The guided task session comprises five tasks of increasing complexity: (1) Concept Q&A; — the participant asks conceptual questions and receives RAG-grounded explanations, testing the Concept QA agent's ability to provide accurate, adaptive responses; (2) Practice Questions — the participant requests exam preparation, answering generated questions and receiving evaluation feedback, testing the Exam Predictor's assessment quality; (3) Weak Point Analysis — the participant reviews their weak points and receives study recommendations, testing the Weak-Point Tracker's metacognitive support; (4) Protégé Mode — the participant teaches a concept to the AI and receives structured feedback, testing the Feynman technique implementation; and (5) Dashboard Review — the participant explores the concept map, mastery dashboard, and flashcard review interface, testing the visualisation and adaptive learning features.

B. Survey Instrument

The survey comprises a 21-item Likert-scale questionnaire (1–5 scale) organised into five sections: Usability (5 items: interface intuitiveness, response clarity, navigation ease, loading speed, error handling), Educational Effectiveness (5 items: explanation quality, practice question relevance, feedback usefulness, concept understanding improvement, study motivation), Visualisation Quality (3 items: concept map clarity, mastery dashboard informativeness, learning path usefulness), Agent Quality (5 items: intent classification accuracy, response relevance, multi-agent coherence, Protégé feedback quality, Viva question appropriateness), and Comparison with Alternatives (3 items: comparison with ChatGPT, comparison with traditional study methods, likelihood of continued use). The System Usability Scale (SUS) [17] is included for standardised benchmarking against the industry average of 68.

C. Quantitative Metrics

Quantitative metrics tracked through the AgentLog system include: average agent response time per agent type (target: < 5 seconds), intent classification accuracy (measured by comparing classified intents against human-annotated ground truth), concept mastery improvement over time (tracked via StudentConcept confidence score deltas between sessions), flashcard retention rates (percentage of due flashcards answered correctly), and study plan adherence (concepts reviewed vs concepts planned). These metrics are accessible through the admin analytics dashboard and can be exported for statistical analysis.

D. Preliminary Results

Initial testing with 5 pilot participants yielded promising qualitative feedback. Participants reported that the multi-agent approach felt more natural than single-chatbot alternatives, as the system automatically adapted its response style based on their query type. The Protégé mode was highlighted as particularly valuable for identifying gaps in understanding that students were not aware of. The concept map visualisation with mastery colouring was noted as an effective motivational tool. Formal user study results with statistical analysis will be reported in a follow-up publication after conducting the full study with 15–20 participants.

XI. Discussion

A. Multi-Agent Architecture Benefits

EduAgent demonstrates that a multi-agent architecture with inter-agent collaboration provides a richer educational experience than single-agent chatbots. The key insight is that different learning activities (explanation,

practice, self-assessment, review, academic writing) benefit from specialised agent behaviours, system prompts, and assessment rubrics. A single system prompt cannot optimally handle the diverse requirements of Socratic questioning for concept exploration, structured rubric evaluation for essay critique, adaptive difficulty for viva examination, and naive probing for Feynman technique teach-back. By specialising agents, EduAgent achieves response quality that would be difficult to replicate with prompt engineering alone in a single-agent system.

B. Inter-Agent Collaboration Effectiveness

The inter-agent collaboration mechanism — where Concept QA considers weak points, Exam Predictor skips mastered topics, and Viva probes known weaknesses — represents a lightweight but effective form of agent communication. Rather than implementing complex agent-to-agent API calls, message queues, or negotiation protocols, EduAgent injects cross-agent context directly into LLM prompts. This approach leverages the LLM's ability to naturally incorporate contextual information into its responses, achieving inter-agent awareness with minimal architectural complexity.

The effectiveness of this approach is evident in the Concept QA agent's behaviour: when a student asks about an advanced topic while having low confidence in its prerequisites, the PREREQUISITE ALERT causes the agent to proactively address foundational gaps before advancing. This behaviour would not emerge naturally from a single-agent system without explicit weak-point awareness.

C. Adaptive Learning Integration

The integration of SM-2 spaced repetition directly into the agent interaction workflow creates a uniquely seamless learning experience. Unlike standalone flashcard tools where students must manually create review items, EduAgent automatically extracts flashcards from conversations and schedules reviews based on demonstrated understanding. The closed-loop feedback cycle (explain $\rightarrow$ check $\rightarrow$ assess $\rightarrow$ update mastery $\rightarrow$ schedule review) ensures that every interaction contributes to the student's long-term retention plan.

D. Limitations

Several limitations should be acknowledged: (1) Dependency on a single LLM provider (Google Gemini) creates a single point of failure. If the Gemini API experiences downtime or rate limiting, all agent functionality is affected. (2) The SM-2 algorithm, while well-established, is relatively simple compared to more sophisticated memory models such as DASH or HL. These newer models account for forgetting curve shape, study context, and individual memory parameters more precisely. (3) Real-time collaboration is currently limited to text-based study rooms. Features such as shared whiteboards, collaborative document editing, or synchronised concept map exploration would enhance the collaborative learning experience. (4) Intent classification relies on LLM-based inference, which introduces latency and potential misclassification for ambiguous queries. A fine-tuned smaller model could reduce this overhead. (5) The evaluation is limited to qualitative pilot testing; a rigorous controlled study with learning outcome measurements is needed to validate educational effectiveness.

E. Future Work

Several directions for future development are identified: (1) Fine-tuning a smaller, faster model (e.g., DistilBERT) for intent classification to reduce latency from the current LLM-based approach. (2) Implementing learning outcome prediction models that can forecast student performance on upcoming assessments based on interaction patterns and mastery trajectories. (3) Adding collaborative whiteboarding features using canvas-based drawing tools with real-time synchronisation via WebSocket. (4) Supporting multiple LLM backends (OpenAI, Anthropic, open-source models) with a provider abstraction layer, eliminating single-provider dependency. (5) Conducting longitudinal studies measuring actual grade improvements over an academic semester. (6) Implementing advanced memory models (DASH, HL) as alternatives to SM-2 for more precise retention scheduling. (7) Adding support for collaborative annotation of course materials within study rooms.

XII. Conclusion

This paper presented EduAgent, a multi-agent AI educational platform that goes beyond single-chatbot solutions by providing six specialised agents with inter-agent collaboration, sharing a unified knowledge layer. The system integrates SM-2 spaced repetition with auto-generated flashcards, AI-powered study planning, voice-enabled oral

examination simulation, multimodal image understanding, collaborative study rooms, custom agent building, and comprehensive analytics.

The key contributions of this work include: a practical implementation of multi-agent educational architecture with prompt-level inter-agent collaboration; seamless integration of spaced repetition algorithms into conversational AI workflows; voice and multimodal interaction modalities that extend beyond text-only tutoring; real-time collaborative learning features with shared AI assistance; and an extensible custom agent framework for professor-defined assessment agents.

The platform is implemented as a production-ready full-stack application with 60+ automated tests, five-service Docker deployment, multi-stage production Dockerfiles, and extensive documentation including deployment guides, demo scripts, and user study protocols. The combination of adaptive learning algorithms, multi-agent AI, real-time collaboration features, and comprehensive analytics demonstrates the potential for AI-driven educational tools that genuinely adapt to individual student needs while providing actionable insights for educators.

EduAgent represents a step toward the vision of AI-powered education that is personalised, adaptive, and collaborative — moving beyond the limitations of single-agent chatbots to provide a holistic learning ecosystem where specialised AI agents work together to support every aspect of the student's educational journey.

References

[1] Khan Academy, "Khanmigo: AI for Education," 2023.

[2] Duolingo, "Duolingo Max with GPT-4," 2023.

[3] Carnegie Learning, "MATHia Adaptive Math Platform," 2023.

[4] S. Hong et al., "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework," arXiv:2308.00352, 2023.

[5] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.

[6] P. Wozniak, "Application of a computer to improve the results obtained in working with the SuperMemo method," 1990.

[7] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.

[8] LangChain, "LangGraph: Build stateful multi-actor applications with LLMs," 2024.

[9] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.

[10] Google, "Gemini: A Family of Highly Capable Multimodal Models," arXiv:2312.11805, 2023.

[11] J. R. Carbonell, "AI in CAI: An Artificial Intelligence Approach to Computer-Assisted Instruction," IEEE Trans. Man-Machine Systems, 1970.

[12] A. Collins, "Processes in Acquiring Knowledge," in Schooling and the Acquisition of Knowledge, 1977.

[13] G. Li et al., "CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society," arXiv:2303.17760, 2023.

[14] J. S. Park et al., "Generative Agents: Interactive Simulacra of Human Behavior," arXiv:2304.03442, 2023.

[15] LangChain, "LangChain: Building applications with LLMs through composability," 2023.

[16] H. Ebbinghaus, "Memory: A Contribution to Experimental Psychology," 1885.

[17] J. Brooke, "SUS: A Quick and Dirty Usability Scale," Usability Evaluation in Industry, 1996.

[18] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.

[19] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL, 2019.

[20] T. Brown et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.